Sensor abstraction через **vtable** — это классический приём из ООП, реализованный на C (без `class`), чтобы работать с разными датчиками через **единый интерфейс**.

Разберём на практике, особенно в контексте твоего ESP32 + IMU 👇

🧠 Идея

Ты хочешь, чтобы код работал одинаково с:

- MPU6050
- MPU9250
- BME280
- и т.д.

👉 Но у них разные функции.

Решение: сделать **общий интерфейс**:

```
sensor_init()
sensor_read()
sensor_get_data()
```

🔧 Что такое vtable

Это просто структура с указателями на функции:

```
typedef struct {
    esp_err_t (*init)(void *self);
    esp_err_t (*read)(void *self);
    esp_err_t (*get_data)(void *self, void *out);
} sensor_vtable_t;
```

👉 Это и есть "виртуальные методы"

🧱 Базовый "класс" сенсора

```
typedef struct {
    const sensor_vtable_t *vtable;
} sensor_t;
```

📦 Конкретный сенсор (пример MPU6050)

```
typedef struct {
    sensor_t base;   // обязательно первым!
    mpu6050_dev_t dev;
} mpu6050_sensor_t;
```

🔌 Реализация функций

```
esp_err_t mpu_init(void *self) {
    mpu6050_sensor_t *s = (mpu6050_sensor_t *)self;
    return mpu6050_init(&s->dev);
}

esp_err_t mpu_read(void *self) {
    mpu6050_sensor_t *s = (mpu6050_sensor_t *)self;
    return mpu6050_get_motion(&s->dev, &s->accel, &s->gyro);
}
```

📚 Таблица виртуальных функций

```
const sensor_vtable_t mpu6050_vtable = {
    .init = mpu_init,
    .read = mpu_read,
    .get_data = mpu_get_data
};
```

🏗 Инициализация объекта

```
mpu6050_sensor_t mpu;

mpu.base.vtable = &mpu6050_vtable;
```

🚀 Использование (магия начинается тут)

```
sensor_t *sensor = (sensor_t *)&mpu;

sensor->vtable->init(sensor);
sensor->vtable->read(sensor);
```

👉 Код НЕ знает, что это MPU6050 😎

🔁 Зачем это нужно

✅ 1. Унификация

```
void process_sensor(sensor_t *s) {
    s->vtable->read(s);
}
```

Работает с любым датчиком

✅ 2. Легко менять железо

```
sensor_t *sensor;

if (use_mpu6050) sensor = &mpu.base;
else sensor = &bme.base;
```

✅ 3. Масштабируемость

Можно сделать массив:

```
sensor_t *sensors[] = { &mpu.base, &bme.base };
```

⚠️ Важные нюансы

1. `void *self`

Это аналог `this` в C++

---

2. base должен быть первым

```
typedef struct {
    sensor_t base;  // ← обязательно первым
```

👉 чтобы каст работал:

```
(sensor_t *)mpu == &mpu.base
```

---

3. Нет type safety

Можно случайно передать не тот тип → будет UB

💡 Упрощённый вариант (без base)

Можно и так:

```
typedef struct {
    const sensor_vtable_t *vtable;
    void *impl;
} sensor_t;
```

🔥 Когда это особенно полезно у тебя

С твоим проектом (IMU + WebSocket):

👉 можно сделать:

```
sensor_t *imu;
sensor_t *mag;
sensor_t *baro;
```

и единый цикл:

```
for (int i = 0; i < N; i++) {
    sensors[i]->vtable->read(sensors[i]);
}
```

🚀 Плюсы vs минусы

✅ Плюсы

- гибкость
- расширяемость
- чистая архитектура
- тестируемость

❌ Минусы

- чуть больше кода
- сложнее дебажить
- нет compile-time проверок

🧠 Аналогия

Это как в C++:

```
class Sensor {
  virtual void read() = 0;
};
```

👉 vtable — это ручная реализация этого механизма

👍 Итог

Sensor abstraction + vtable — это:

- способ сделать "ООП на C"
- отделить логику от конкретного железа
- писать универсальный код для датчиков